

Does Point Cloud Upsampling Help Downstream Classification?

A unified $\times 4$ two-line protocol on ModelNet40 with from-scratch PointNet++ classifiers, equal-cardinality geometry evaluation, and a mesh-reference control experiment

5

upsampling methods
compared under one protocol

14

PointNet++ classifiers
trained from scratch

12,311

ModelNet40 shapes
processed per method and line

2,468

test shapes in the
equal-N geometry evaluation

One-sentence finding, and the four numbers behind it

Under a unified $\times 4$ protocol, upsampling improves geometric agreement with a reference surface but does not improve — and usually degrades — PointNet++ shape classification.

0 / 5

Line A — densification

No 4096-point upsampled branch beats the native Original 1024 baseline of 91.95 %. The strongest, PU-GCN, is 0.32 pp below it.

+0.42 pp

Line B — recovery

Only PU-Net exceeds the sparse 256-point baseline. It remains 0.68 pp below Original 1024.

+0.05 pp

Mesh-ref control

A genuine mesh-sampled 4096-point cloud beats Original 1024 by 0.05 pp — point count alone explains nothing.

rank flip

Geometry $\neq$ accuracy

PU-GCN attains the lowest Chamfer Distance in both lines, yet PU-Net beats it on accuracy in Line B.

Methodological by-product: the Chamfer ranking of the five methods reverses with the choice of reference point set. A degenerate control that merely repeats each input point four times scores a perfect $CD = HD = 0.0000$ under the superseded reference, and last place under the equal-cardinality protocol adopted here.

What this report found — six results and their evidence

Everything after this slide is the evidence for these six rows. Sections III and IV show how each was established and what had to be corrected on the way.

Finding	Direct evidence	Implication for the thesis
Geometric quality does not predict classification utility	Line B: PU-GCN has the lowest CD (0.0558) yet 90.06 % OA; PU-Net has the highest CD (0.0646) yet 91.27 %.	Geometry and task metrics must be reported in separate tables; neither substitutes for the other.
Densification provides no benefit	Line A: all five methods fall 0.32 to 1.41 pp below the Original 1024 baseline of 91.95 %.	The “denser is better” assumption does not hold for adequately sampled CAD shapes.
Recovery is bounded and method-specific	Line B: PU-Net recovers 0.42 of the 1.10 pp decimation loss; the other four make the result worse.	Upsampling cannot reconstruct information that decimation destroyed.
Distribution matters more than mean error	PU-Net: best HD and $\Delta\text{NUC} \approx 0$, and the only accuracy gain. PU-GCN: NUC 41 % above Original, -1.21 pp against PU-Net.	PointNet++ ball-query grouping is sensitive to local density regularity and to outliers.
Point count is not the causal variable	Mesh-ref 4096 beats Original 1024 by only $+0.05$ pp; mesh-ref 256 matches Downsampled 256 to within 0.03 pp.	Line A deficits must be attributed to how the points were generated, not to how many there are.
Densification has no channel to help	PointNet++ SA1 caps each neighbourhood at 32 points: it already discards 31.9 % of them at 1024 and 78.7 % at 4096.	The upside is structurally unavailable while the error introduced remains visible, so the expected value of upsampling here is at most zero.

Report structure

I**Problem and Design**

Research questions, the five upsampling methods, the two-line $\times 4$ protocol, and the point-count invariants that make the comparison valid.

II**Evaluation Protocol**

Definitions of CD / HD / NUC together with their implementation sites, and the equal-N mesh-reference construction.

III**Results**

Classification for Line A, Line B and the mesh-ref control; equal-N geometry; geometry–accuracy decoupling; qualitative evidence.

IV**Problems and Analysis**

Ten problems: the reference-frame defect that changed a conclusion, four protocol confounds caught by audit, dataset and documentation defects, cluster failures, and an external challenge to the headline result.

V**Status and Outlook**

Protocol evolution, limitations, conclusions, and the material index for the thesis chapter.



Problem Formulation and Experimental Design

What is actually being tested, and how the comparison is kept fair

- Research questions RQ1–RQ4 and what would falsify each
- The five upsampling methods, their families and their operating principles
- The two-line $\times 4$ protocol: densification versus recovery
- Five point-count invariants separating the upsampling effect from the point-count effect

The untested assumption in the upsampling literature

The gap

Point cloud upsampling generates a denser, geometrically more complete point set from a sparse input. The field evaluates the result almost exclusively with surface-fidelity metrics — Chamfer Distance, Hausdorff Distance, point-to-face distance — and then asserts, implicitly, that a denser and geometrically better point cloud benefits downstream 3-D tasks.

That last step is rarely measured. This study tests it directly: the upsampled point clouds are fed to a PointNet++ classifier trained from scratch, and the change in Overall Accuracy is reported alongside geometric metrics computed on the same shapes.

RQ1 **Densification**

Does $\times 4$ upsampling of an already well-sampled Original 1024 cloud to 4096 points improve PointNet++ accuracy?

RQ2 **Recovery**

After $\times 4$ decimation to 256 points, can upsampling back to 1024 recover the lost classification accuracy?

RQ3 **Metric alignment**

Is the method with the best geometric score also the best method for the downstream task?

RQ4 **Point-count confound**

Is any observed change explained simply by the number of points, independently of how those points were produced?

Five upsampling methods under comparison

All upsamplers run as frozen pre-trained inference. No upsampling network was re-trained in this study.

Method	Family	Operating principle	Reference
EAR	Optimisation (non-learning)	Edge-Aware Resampling. Resamples away from sharp features first so that normals can be estimated reliably, then progressively pushes points toward the edges. Deterministic; no checkpoint.	Huang et al., ACM TOG 2013
PU-Net	Learned (TensorFlow 1)	The first learned upsampler. Multi-level point feature extraction, then expansion in feature space through parallel convolution branches, followed by coordinate reconstruction.	Yu et al., CVPR 2018
PU-GCN	Learned (TensorFlow 1)	Graph-convolutional upsampling. Inception-DenseGCN encodes multi-scale local neighbourhood geometry and NodeShuffle expands the point set in the graph domain. Typically strong on CD.	Qian et al., CVPR 2021
PDANS	Learned (PyTorch)	Conditional diffusion upsampling with Point Diffusion Adaptive Noise Suppression and a tree-structured Transformer, designed for robustness to input noise while preserving fine detail.	CVPR 2025 (Baty2023/PDANS)
PU-EdgeFormer	Learned (TensorFlow)	Hybrid EdgeFormer block: EdgeConv captures local graph structure while multi-head self-attention supplies global context, combining both receptive fields in a single operator.	Kim et al., ICASSP 2023

Excluded from the main comparison: TULIP (LiDAR range-image upsampling). Its output format is incompatible with the xyz point-count protocol used here, so it is retained as supplementary material only.

Classifier, dataset, and the definition of the reported accuracy

PointNet++ (SSG) — the downstream model

A hierarchical extension of PointNet. Each set-abstraction layer selects centroids by farthest-point sampling, groups their neighbours with a ball query, and applies a shared PointNet MLP to each local region, yielding a coarser but semantically richer point set. Stacking such layers builds a local-to-global feature hierarchy.

SSG denotes single-scale grouping (one radius per layer), as opposed to the multi-scale MSG variant. Input features here are XYZ coordinates only — no normals.

ModelNet40 — the dataset

12,311 CAD meshes across 40 object categories (Wu et al., CVPR 2015), used with the official split of 9,843 train and 2,468 test shapes. Point clouds are obtained by area-weighted triangle sampling of the .off mesh followed by unit-sphere normalisation.

The standard ModelNet40 classification setup has no validation split: both the original PointNet++ code and this implementation select the best checkpoint on the test set. This is stated explicitly rather than hidden.

Reported accuracy — two definitions, kept separate

Quantity	Definition	Role in this report
Best Overall Accuracy	Highest instance-level (overall) test accuracy recorded across the 200 training epochs; the checkpoint is written whenever this value improves.	Primary metric throughout
Final Overall Accuracy	Overall test accuracy at the last (200th) epoch, whether or not it was the maximum.	Supplementary, reported alongside
Δ in pp	Difference between two Best Overall values, expressed in percentage points — never as a relative percentage.	All comparisons

IMPLEMENTATION

```
scripts/train_pointnet2.py:322-328
    if overall_acc >= best_overall: best_overall = overall_acc ; torch.save(...)
```

The two-line $\times 4$ protocol

Two independent experimental lines. Cross-line comparison is deliberately not treated as a primary result.

LINE A · DENSIFICATION

"Good data, made denser" — does adding points to an already adequate cloud help?



Baseline for Line A: the Original 1024 cloud classified at its native 1024 points (91.95 % Best OA).

LINE B · RECOVERY

"Degraded data, restored" — can upsampling undo a $\times 4$ loss of points?



Baseline for Line B: the Downsampled $\times 4$ cloud classified at its native 256 points (90.85 % Best OA).

Each of the 12 branches above, plus 2 mesh-reference controls, is a separate PointNet++ trained from scratch for its own point count — not a 1024-point model evaluated on more points.

Five point-count invariants

These rules exist so that the measured effect is the upsampling method and not the number of points.

1

Baselines keep their native cardinality

A baseline is never upsampled or padded to match an upsampled branch. Original stays at 1024; Downsampled stays at 256.

2

Upsampled outputs keep their output cardinality

An upsampled cloud is never cropped back to the baseline size in order to “make the comparison fair”.

3

No artificial baseline ↔ upsampling point matching

Any such alignment silently merges the upsampling effect with the point-count effect and renders the result uninterpretable.

4

One line = one ratio = one output cardinality

All methods within a line share the same ×4 ratio and emit the same number of points, so they are mutually comparable.

5

Line A and Line B are independent

They answer different questions. A number from Line A never appears in the same ranking as a number from Line B.

Why it matters

If a 256-point baseline is resampled up to 1024 inside the data loader, it will appear to “improve” for reasons unrelated to any upsampling network.

That configuration was in fact produced early in this project (Step 6). It is now retained only as a negative-control ablation, never as a baseline.

ENFORCED IN CODE

```
scripts/train_pointnet2.py:50
--no-allow-resample
```

Rejects any sample whose on-disk point count differs from `num_point`. Silent crop or pad is impossible.

Training configuration — identical across all 14 runs

Parameter	Value	Parameter	Value
Model	pointnet2_cls_ssg	Epochs	200
Classes	40	Optimiser	Adam
Input features	XYZ only (no normals)	Learning rate	0.001
Batch size	24	Weight decay	0.0001
Random seed	42	Dataloader workers	4
Train / test split	9,843 / 2,468 (official)	allow_resample	false

The 14 training runs

Line A — 6 runs

Original 1024 baseline plus 5 methods at 4096

Line B — 6 runs

Downsampled 256 baseline plus 5 methods at 1024

Controls — 2 runs

Mesh-ref 256 and mesh-ref 4096

Data provenance

Artefact	Path	Construction
Original 1024	datasets/modelnet40_original/	Area-weighted mesh sampling plus unit-sphere normalisation
Downsampled ×4	datasets/modelnet40_downsampled_x4/	256 points decimated from Original 1024, 12,311 shapes
Mesh-ref 256 / 4096	datasets/modelnet40_mesh_ref_{256,4096}/	Independently re-sampled from the same .off meshes
Upsampled outputs	datasets/line{A,B}_*/strict_*/<method>/	Strict point-count audit, zero NaN or Inf, 12,311 shapes per method

SECTION

II

Evaluation Protocol

How geometry is measured, and why the reference set had to be rebuilt

- Exact definitions of Chamfer Distance, Hausdorff Distance and NUC, with code sites
- The diagnosis: why scoring 4096 points against a 1024-point reference is unsound
- A degenerate control — an output that does nothing scores perfectly on the old metric
- The remedy: mesh-reference point sets that make every comparison equal-N

Geometric quality — definitions as implemented

CD**Chamfer Distance**

The sum of the two mean nearest-neighbour distances between point sets A and B: the mean over a in A of $\text{dist}(a, B)$, plus the mean over b in B of $\text{dist}(b, A)$. Symmetric. Lower is better.

Sensitive to systematic offsets; insensitive to a small number of outliers.

HD**Hausdorff Distance**

The maximum of the two worst-case nearest-neighbour distances: $\max(\max_a \text{dist}(a, B), \max_b \text{dist}(b, A))$. Lower is better.

Governed entirely by the single worst point — an outlier detector rather than an average-quality measure.

NUC**Non-Uniformity Coefficient**

128 query centres are drawn from the cloud; at each of three radii (0.02, 0.05, 0.10) the neighbour counts are collected and their coefficient of variation std/mean is taken. NUC is the mean of the three values.

A distribution-uniformity measure, not a fidelity measure. Comparable within this protocol only.

IMPLEMENTATION

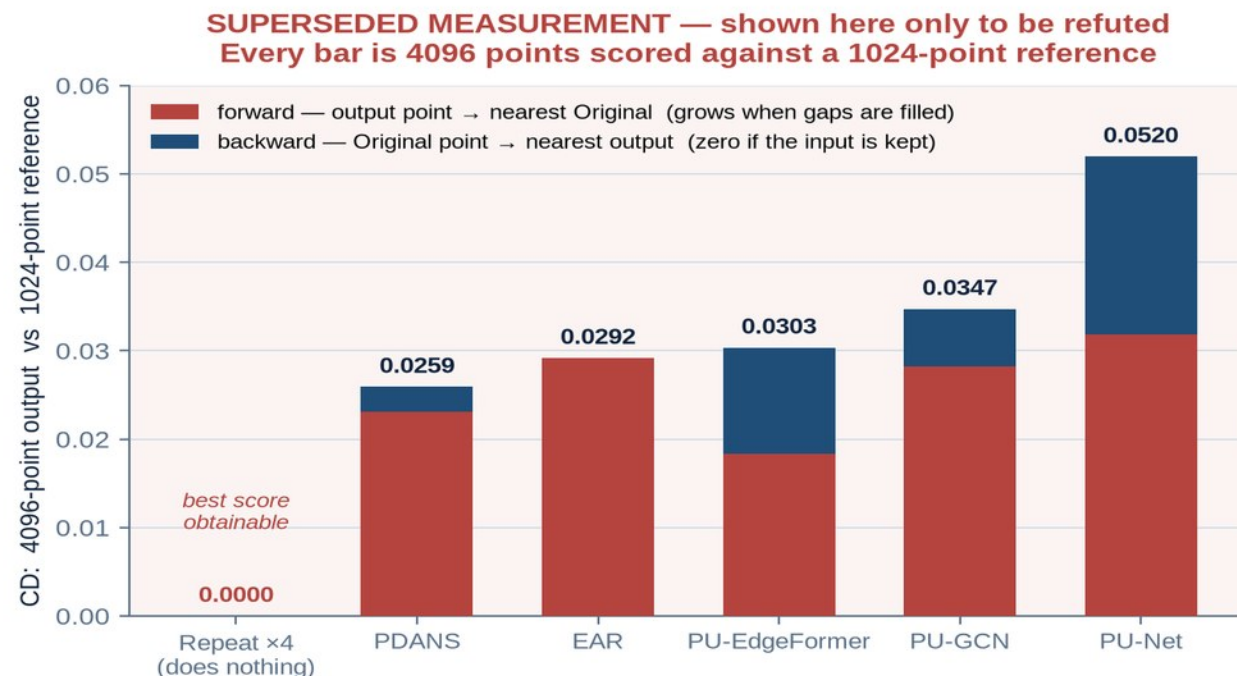
```
scripts/  
  compute_geometry_equal_n.py  
  
:63 nn_distances()  
  cKDTree.query(k=1)  
  
:69 chamfer_hd()  
  cd = mean(fwd)+mean(bwd)  
  hd = max(max_fwd, max_bwd)  
  
:82 compute_nuc()  
:36 NUC_QUERY_CENTERS = 128  
:37 NUC_RADII=(.02, .05, .10)
```

REPORTED AS

Per-shape values are computed for all 2,468 test shapes and then averaged. Standard deviations accompany every mean in the summary CSV, and the full per-sample table is retained.

Why the reference must have the same number of points

Every figure on this page is the measurement this project abandoned. It is shown so the abandonment can be justified, not as a result.



THE REFERENCE WAS ALSO THE INPUT

Line A upsamples Original 1024 to 4096, so scoring that output against Original 1024 asks how little the method disturbed its own input. The forward term grows precisely when a method fills the gaps between input points; the backward term falls to zero whenever those points are simply kept in place.

MEASURED

The forward term supplies 61–100 % of every old CD score. EAR's backward term is exactly 0.00000000: it keeps every input point verbatim.

The control settles it. A cloud that merely repeats each input point four times performs no upsampling whatsoever, yet attains $CD = HD = 0.0000$ against Original 1024 — the best value obtainable — while ranking last of all six variants under equal-N ($CD\ 0.0557$). A metric that awards a do-nothing output the top score is measuring the wrong quantity. Note this also means the superseded metric could never have detected a repeat- $\times R$ fake upsampler; only the dedicated authenticity audit could.

Equal-N evaluation and the mesh-reference construction

The fix for the defect on the previous page: never compare two point sets of different size again. Every geometric number after this page obeys it.

The problem

Comparing a 4096-point cloud against a 1024-point reference measures discrete-set consistency between two differently sized samples of the same surface. It is not a measure of surface accuracy, and it systematically favours whichever output cardinality lies closer to the reference.

The fix

For every shape, sample an independent reference point set of exactly the cardinality being evaluated, directly from the source .off mesh rather than by decimating the Original cloud. CD and HD are then always computed between two sets of equal size.

Reference construction

- Source: ModelNet40 .off meshes, resolved via the original manifests
- Area-weighted triangle sampling, as used to produce Original 1024
- Unit-sphere normalisation applied afterwards
- Per-shape deterministic seed, so the reference is reproducible
- 12,311 shapes at both 256 and 4096; the build audit reports 0 failures

IMPLEMENTATION

```
build_mesh_reference_points.py:66
seed = stable_seed(42, "mesh_ref", str(N), key)
:80 points = normalize_unit_sphere(points)
```

What is compared against what

Evaluated set	Points	Reference
Line A upsamplers	4096	Mesh-ref 4096
Line B Downsampled ×4	256	Mesh-ref 256
Line B upsamplers	1024	Original 1024
<i>Mesh-ref / Original (self)</i>	—	<i>CD = HD = 0 by definition</i>

SECTION

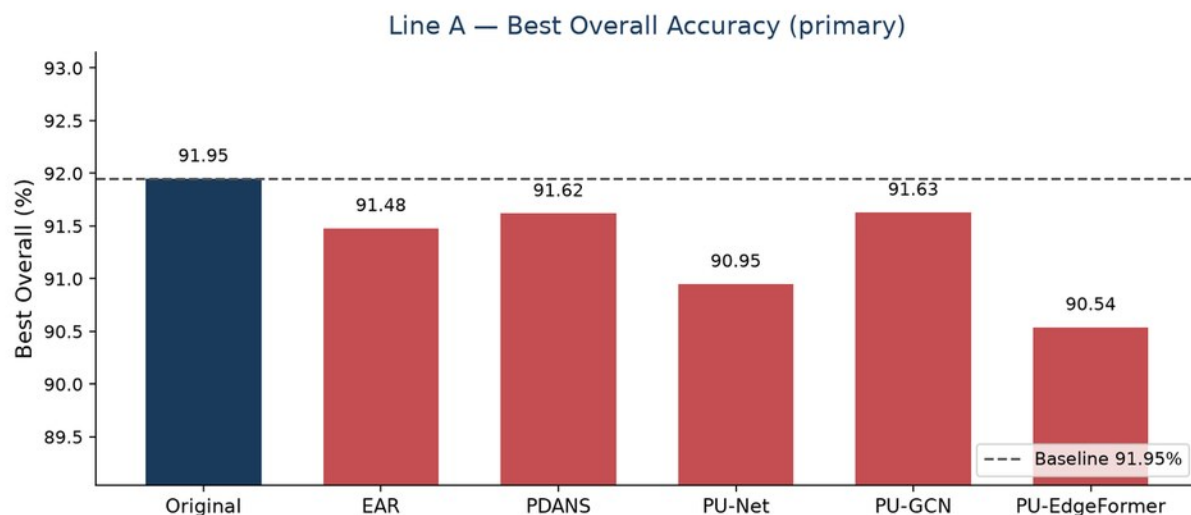


Results

Classification, geometry, and the relationship between them

- Line A and Line B classification accuracy, plus the mesh-reference control
- The same evidence regrouped by point count: 256, 1024 and 4096 compared separately
- Direct evidence that the geometric ranking and the accuracy ranking disagree
- Why the synthetic clouds lose to the genuine one, measured at PointNet++'s own layer
- Qualitative comparison under a fixed camera, axis range and point-size rule

Densification does not improve classification

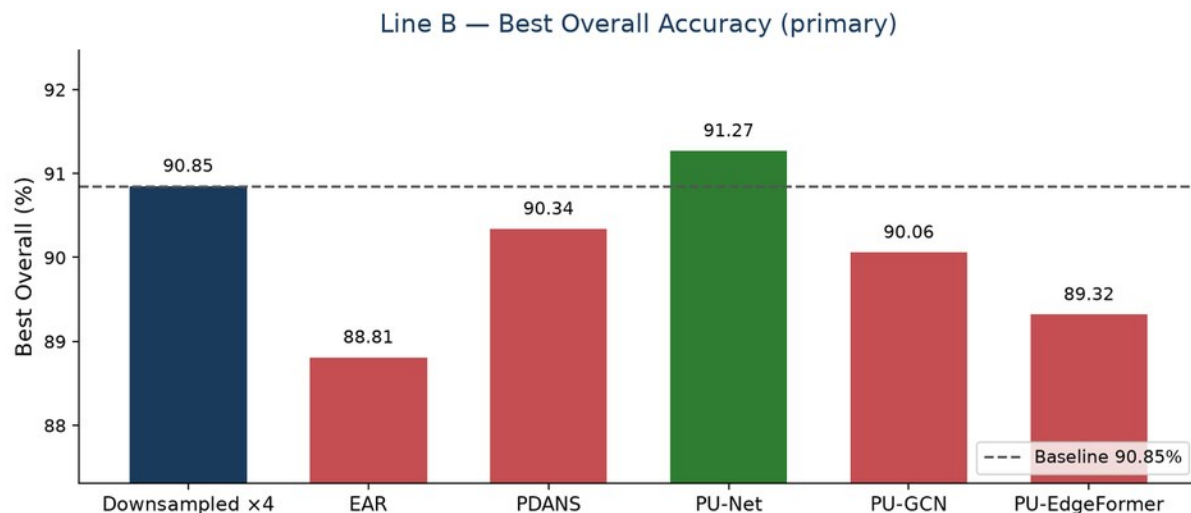


Method	Pts	Best OA	Final OA	Best CIs	Δ pp
Original baseline	1024	91.95 %	91.31 %	88.00 %	0.00
PU-GCN	4096	91.63 %	91.00 %	87.74 %	-0.32
PDANS	4096	91.62 %	90.87 %	88.44 %	-0.33
EAR	4096	91.48 %	90.85 %	88.19 %	-0.47
PU-Net	4096	90.95 %	90.55 %	87.46 %	-1.00
PU-EdgeFormer	4096	90.54 %	90.13 %	86.35 %	-1.41

READING

No 4096-point branch reaches the Original 1024 baseline. The spread among methods (0.32 to 1.41 pp below baseline) is larger than the spread among their Chamfer Distances, and its ordering does not follow the CD ordering. PU-EdgeFormer loses most on both Overall (-1.41 pp) and Class accuracy (-1.65 pp), while PU-GCN and PDANS are almost indistinguishable from one another. RQ1 is therefore answered negatively: for an already adequately sampled CAD point cloud, $\times 4$ densification does not translate into PointNet++ gains.

Recovery is partial, and only for one method



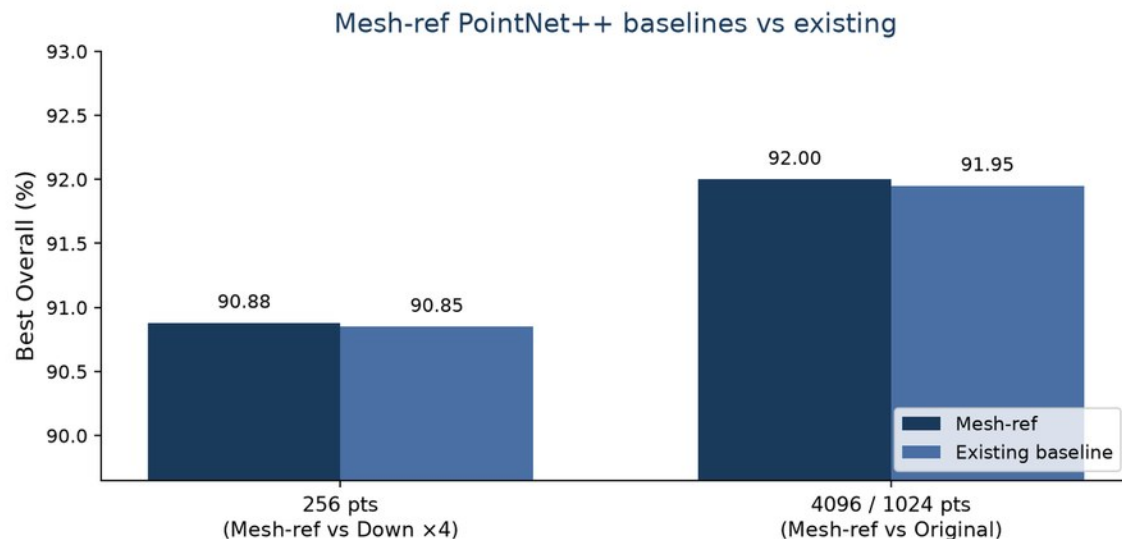
Method	Pts	Best OA	Final OA	Δ vs Down	gap vs Orig
Downsampled x4	256	90.85 %	90.46 %	0.00	-1.10
PU-Net	1024	91.27 %	90.65 %	+0.42	-0.68
PDANS	1024	90.34 %	89.46 %	-0.51	-1.61
PU-GCN	1024	90.06 %	89.63 %	-0.79	-1.89
PU-EdgeFormer	1024	89.32 %	88.58 %	-1.53	-2.63
EAR	1024	88.81 %	88.02 %	-2.04	-3.14

READING

Restoring the point count does not restore the discriminative content. Only PU-Net improves on the 256-point baseline, by +0.42 pp, and it remains 0.68 pp below Original 1024 — so at best about 38 % of the decimation loss is recovered. Four of the five methods are worse than simply classifying the sparse 256-point cloud directly. EAR, the classical optimisation method, is weakest by a wide margin (-2.04 pp) despite producing visually plausible surfaces. RQ2 is answered: upsampling cannot substitute for information that decimation removed.

Mesh-reference control: is the point count itself responsible?

RQ4. Two additional PointNet++ classifiers trained from scratch on independently mesh-sampled clouds.



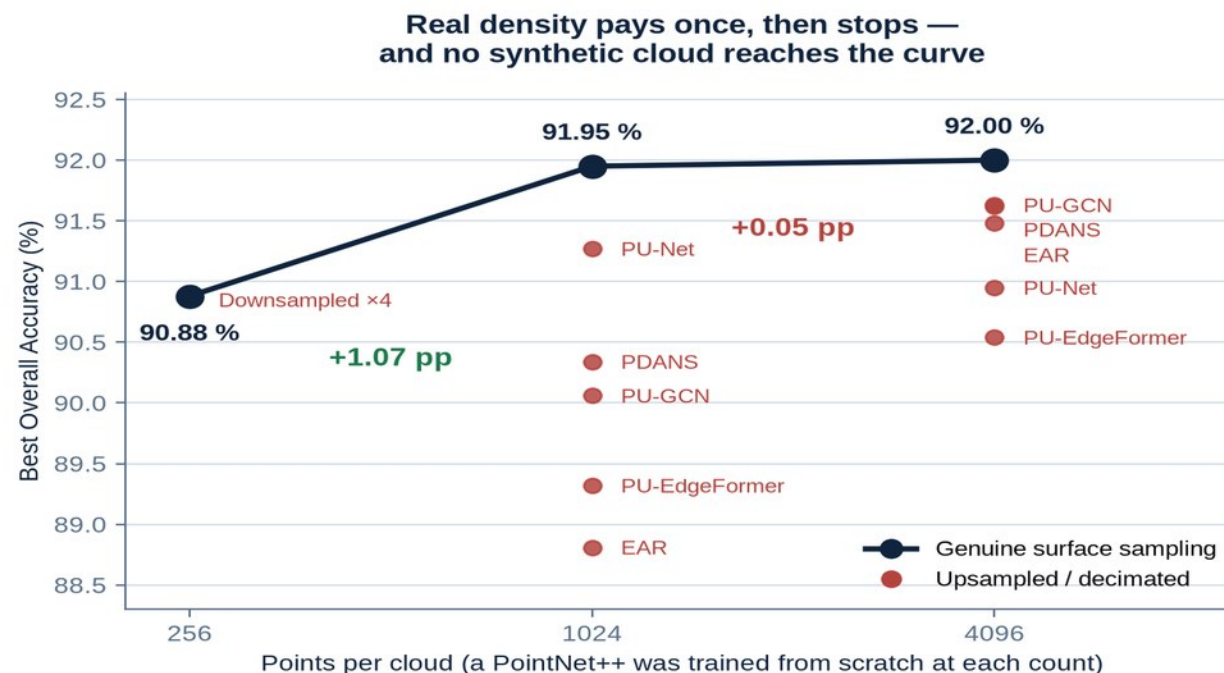
Control branch	Pts	Best OA	Best ep.	Compared with	Δ pp
Mesh-ref 256	256	90.88 %	129	Downsampled $\times 4$ 256 (90.85 %)	+0.03
Mesh-ref 4096	4096	92.00 %	148	Original 1024 (91.95 %)	+0.05

Quadrupling the point count — with points drawn honestly from the true surface — buys 0.05 pp. Point count is therefore not a meaningful explanatory variable in this setting.

Consequence for the interpretation of Line A. Because a genuine 4096-point sampling of the surface performs the same as Original 1024, the 0.32 to 1.41 pp deficits of the Line A upsamplers cannot be attributed to the change in cardinality, to the resulting change in PointNet++ receptive-field statistics, or to the sampling procedure as such. They must be attributed to the geometric and distributional characteristics of the generated points themselves.

Three point counts, three self-contained comparisons

Inside a bucket every cloud carries the same cardinality, so CD / HD are equal-N — and every accuracy comes from a PointNet++ trained at that cardinality.



THE RETURN ON DENSITY COLLAPSES AT 1024

Quadrupling from 256 to 1024 is worth +1.07 pp. Quadrupling again to 4096 is worth +0.05 pp — twenty times less, for the same factor of four. Whatever an upsampler might contribute at 4096, the task itself has almost nothing left to give.

256-point bucket	CD ↓	HD ↓	NUC	Best OA
Mesh-ref 256 (genuine)	0.0000	0.0000	2.1265	90.88 %
Downsampled ×4 (decimated)	0.1333	0.2049	2.0933	90.85 %

Decimating 1024 → 256 costs nothing beyond the point count itself (−0.03 pp).

In every bucket the genuine cloud wins. At 256 the two constructions tie. At 1024 the best reconstruction is 0.68 pp below the real cloud; at 4096 the best upsampling is 0.37 pp below a real mesh sampling. No synthetic cloud has ever matched, let alone beaten, the genuine one at its own cardinality.

The 1024-point bucket — genuine data vs reconstruction

All seven clouds hold 1024 points, are scored against Original 1024, and were classified by a PointNet++ trained at 1024.

1024-point cloud	Origin	CD ↓	HD ↓	NUC	ΔNUC	Best OA	vs genuine
Original 1024	<i>genuine sampling</i>	0.0000	0.0000	1.0836	0.000	91.95 %	—
PU-Net	256 → ×4	0.0646	0.1083	1.0801	−0.003	91.27 %	−0.68
PDANS	256 → ×4	0.0627	0.1905	1.0528	−0.031	90.34 %	−1.61
PU-GCN	256 → ×4	0.0558	0.1514	1.5240	+0.440	90.06 %	−1.89
PU-EdgeFormer	256 → ×4	0.0619	0.1776	1.1225	+0.039	89.32 %	−2.63
EAR	256 → ×4	0.0767	0.1932	0.9653	−0.118	88.81 %	−3.14

GEOMETRY DOES NOT ORDER THE COLUMN ON THE RIGHT

PU-GCN has the best CD in this bucket and finishes fourth on accuracy. PU-Net has the worst CD of the four learned methods and finishes first. The two quantities that do track accuracy here are HD and $|\Delta\text{NUC}|$ — outliers and local density regularity.

THE COST OF RECONSTRUCTION

Every reconstruction is worse than the genuine 1024-point cloud, by between 0.68 and 3.14 pp. Restoring the count restores roughly 38 % of what decimation removed, at best; the rest is not recoverable from 256 points.

The 4096-point bucket — genuine data vs upsampling

All six clouds hold 4096 points, are scored against mesh-ref 4096, and were classified by a PointNet++ trained at 4096.

4096-point cloud	Origin	CD ↓	HD ↓	NUC	ΔNUC	Best OA	vs genuine
Mesh-ref 4096	<i>genuine sampling</i>	0.0000	0.0000	0.6419	0.000	92.00 %	—
PU-GCN	1024 → ×4	0.0423	0.0932	0.5204	−0.121	91.63 %	−0.37
PDANS	1024 → ×4	0.0468	0.1135	0.6612	+0.019	91.62 %	−0.38
EAR	1024 → ×4	0.0547	0.1154	0.8156	+0.174	91.48 %	−0.52
PU-Net	1024 → ×4	0.0497	0.1112	0.6052	−0.037	90.95 %	−1.05
PU-EdgeFormer	1024 → ×4	0.0467	0.1109	0.6754	+0.034	90.54 %	−1.46

THE RANKING FLIPS BETWEEN THE TWO BUCKETS

PU-Net is first at 1024 and fifth at 4096; PU-GCN is fourth at 1024 and first at 4096. No method is uniformly best, which is why densification and recovery have to be reported as separate questions rather than one leaderboard.

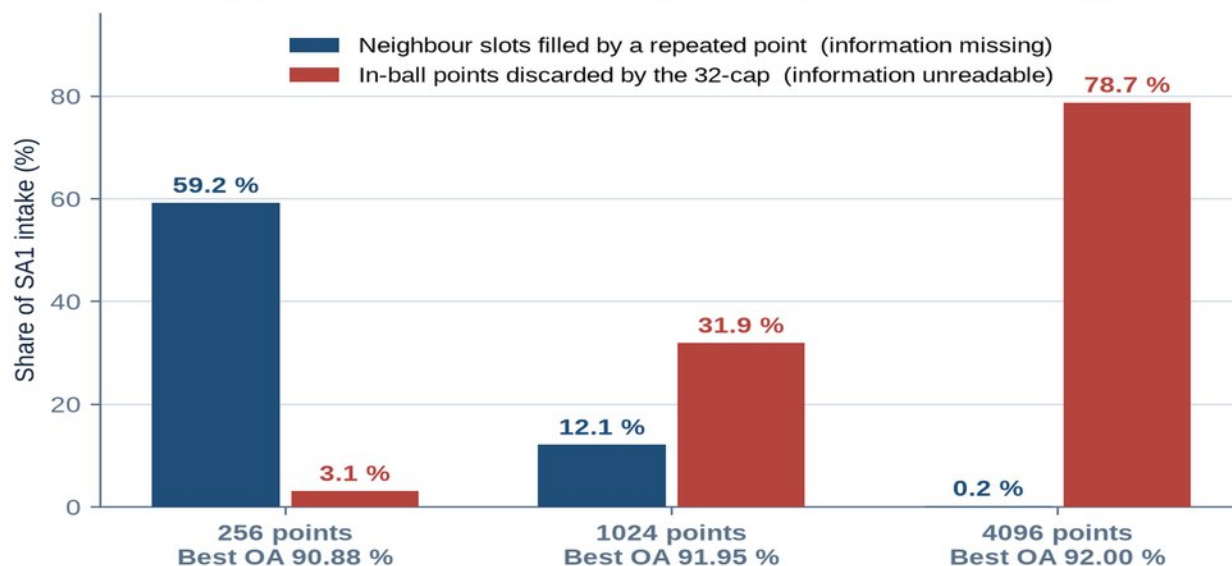
AND THE WHOLE BUCKET IS BOUNDED ABOVE

Even the genuine 4096-point cloud is only +0.05 pp above Original 1024. The entire prize available in this bucket is five hundredths of a point, and every upsampler lands 0.37 to 1.46 pp on the wrong side of it.

Why the synthetic clouds lose, measured at PointNet++'s own first layer

SA1 is $npoint = 512$, $radius = 0.2$, $nsample = 32$, and it consumes the raw input cloud — so what it can and cannot see is directly measurable.

The two ways a cloud fails PointNet++ SA1 trade places with density ($npoint = 512$, $radius = 0.2$, $nsample = 32$ — genuine clouds only)



A FIXED READING CAPACITY OF 32

More than 32 points inside the radius: the surplus is discarded. Fewer than 32: the empty slots are filled by repeating the first neighbour. Density does not change that budget.

IMPLEMENTATION

```
models/pointnet2_utils.py
:: query_ball_point

group_idx[sqrddists>r**2] = N
group_idx = sort(...)[:, :, :nsample]
group_idx[mask] = group_first[mask]
```

This is the shape of the density curve. Going 256 → 1024 removes the padding that was filling 59.2 % of every neighbourhood, and buys +1.07 pp. Going 1024 → 4096 has almost no padding left to remove (12.1 % → 0.2 %) while the discarded share rises from 31.9 % to 78.7 % — points the layer structurally cannot read. Hence +0.05 pp for an honest 4096 cloud, and no headroom for any upsampler to win there.

Per-variant SA1 intake inside each point-count bucket

Comparable only within a bucket: the intake statistics are dominated by the point count, which is exactly why the buckets are kept apart.

Input cloud	Role	Pts	Mean in ball	Discarded	Repeated slots	Density CV	kept/random spread	Best OA
Mesh-ref 256	<i>genuine</i>	256	14.5	3.1 %	59.2 %	0.407	1.000	90.88 %
Downsampled ×4	<i>decimated</i>	256	14.5	3.1 %	59.2 %	0.406	1.000	90.85 %
Original	<i>genuine</i>	1024	54.8	31.9 %	12.1 %	0.363	1.001	91.95 %
PU-Net	<i>best at 1024</i>	1024	55.1	31.3 %	10.8 %	0.333	1.001	91.27 %
PDANS		1024	58.7	36.1 %	9.0 %	0.379	1.025	90.34 %
PU-GCN		1024	52.7	30.7 %	13.1 %	0.374	1.016	90.06 %
PU-EdgeFormer		1024	57.6	35.9 %	11.1 %	0.417	0.985	89.32 %
EAR	<i>worst at 1024</i>	1024	72.3	53.0 %	14.5 %	0.645	1.054	88.81 %
Mesh-ref 4096	<i>genuine</i>	4096	213.4	78.7 %	0.2 %	0.352	1.002	92.00 %
PU-GCN	<i>best at 4096</i>	4096	211.1	78.1 %	0.1 %	0.334	1.026	91.63 %
PDANS		4096	215.1	78.9 %	0.1 %	0.359	1.034	91.62 %
EAR		4096	234.5	81.7 %	2.1 %	0.661	1.086	91.48 %
PU-Net		4096	216.6	79.2 %	0.1 %	0.319	0.997	90.95 %
PU-EdgeFormer	<i>worst at 4096</i>	4096	222.2	79.8 %	0.2 %	0.365	0.970	90.54 %

EAR is the clearest case. Its density CV is 0.645 at 1024 against 0.33–0.42 for everything else, and it is simultaneously the most saturated (53.0 % discarded) and among the most starved (14.5 % repeated slots) — concentrating points on edges wastes them where they pile up and starves the layer where they do not. It is last in the 1024 bucket.

What the measurements rule out, and what stays open

Reported so the explanation is not stronger than the evidence behind it.

● Ruled out — truncation is not spatially biased

Because the 32 survivors are chosen by storage index, an upsampler whose output order correlated with position would hand the layer a clustered sample. Measured: the spatial spread of the kept 32 divided by that of 32 random in-ball points is 0.97–1.09 across all variants. The surviving subset is spatially representative, so this is not the mechanism.

● Ruled out — point count is not the cause

The mesh-reference controls match their counterparts at 256 (+0.03 pp) and at 4096 (+0.05 pp). Cardinality by itself neither helps nor harms; only how the points were produced does.

● Supported — capacity saturation bounds the upside

SA1 discards 31.9 % of each neighbourhood at 1024 and 78.7 % at 4096, while the repeated-point padding that dominates 256 (59.2 % of slots) is all but gone by 1024. The accuracy spread between methods contracts in step, from 2.46 pp in the 1024 bucket to 1.09 pp in the 4096 bucket.

● Supported — outliers and density regularity, not mean error

In the 1024 bucket, Spearman rank correlation with Best OA is +0.70 for HD and +0.70 for $|\Delta\text{NUC}|$, but only +0.10 for CD. Worst-case error and local uniformity track the task; average surface error does not.

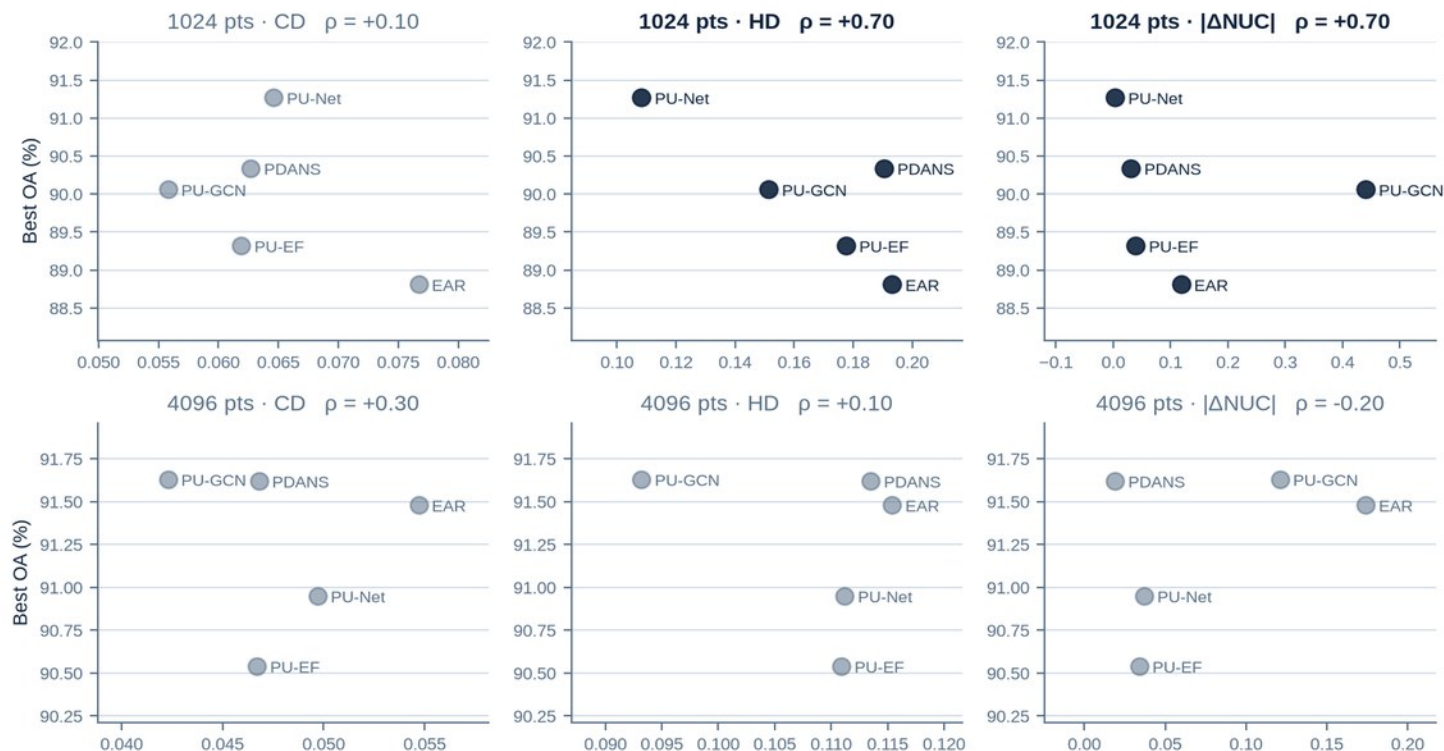
● Open — no statistic orders the 4096 bucket

There the same correlations are +0.30 (CD), +0.10 (HD) and -0.20 ($|\Delta\text{NUC}|$). Once the layer saturates, the remaining differences are small and no geometric measure explains their order. With $n = 5$ methods and one seed per branch, these coefficients are descriptive only.

Which geometric measure, if any, predicts the task

Rank correlation between each equal- N geometric measure and Best Overall Accuracy, computed separately inside each point-count bucket.

Spearman rank correlation with accuracy, per point-count bucket ($n = 5$ methods; descriptive only)



AT 1024 — TWO MEASURES TRACK

HD and $|\Delta\text{NUC}|$ both reach $\rho = +0.70$ with accuracy, while CD manages only $+0.10$. Worst-case error and local density regularity track the task; average surface error does not.

AT 4096 — NOTHING TRACKS

The same correlations fall to $+0.30$, $+0.10$ and -0.20 . Once SA1 saturates, the remaining spread is only 1.09 pp and no geometric measure explains its order. $n = 5$ per bucket with one seed: descriptive, not inferential.

A geometric benchmark cannot stand in for downstream utility — and which geometric measure is even worth reporting depends on the point count.

Densification — same shape, same camera, same axis limits

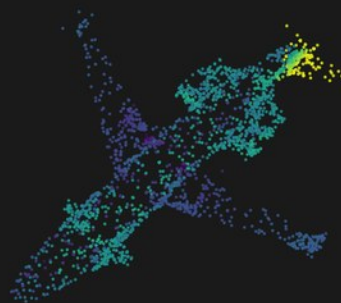
Line A · airplane · airplane_0627

Same sample · Same camera · Same axis limits · Same point-size rule | Color = depth (not error)

Original
1024 pts



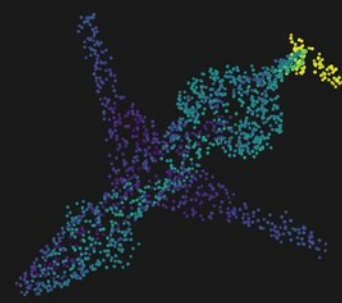
EAR
4096 pts



PDANS
4096 pts



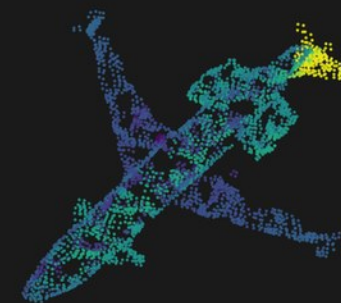
PU-Net
4096 pts



PU-GCN
4096 pts



PU-EdgeFormer
4096 pts



EAR

Points migrate toward the wing and fuselage edges, thinning the interior surface — visible as sparse patches on the wings.

PDANS

Dense and even, but with a faint lattice texture inherited from the diffusion sampling grid.

PU-Net

Follows the input distribution closely; the least visually altered relative to Original.

PU-GCN

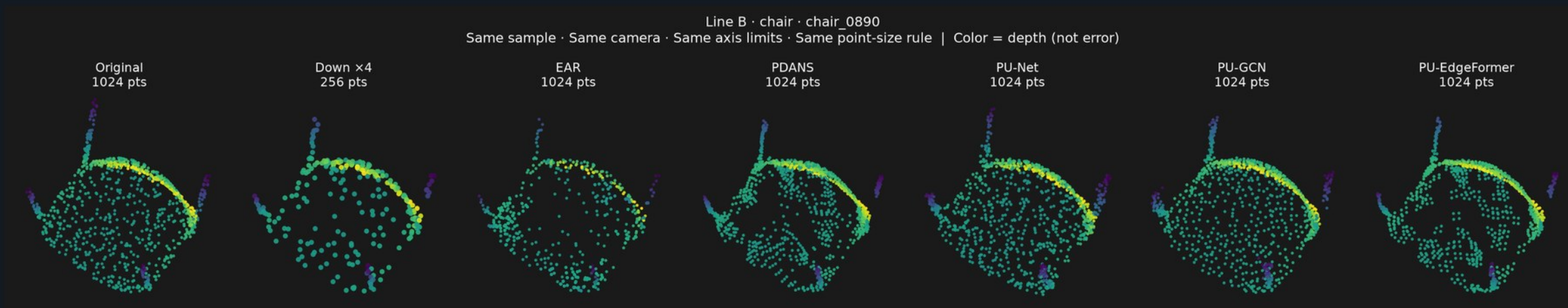
The smoothest and most surface-like result, and the one that most resembles a genuine dense sampling.

PU-EdgeFormer

Similar in character to PDANS, with slightly heavier clustering near the nose and tail.

Colour encodes depth, not error. Identical elevation, azimuth, axis range and point-size rule across all six panels.

Recovery — what 256 points lose, and what upsampling puts back



What the eye sees

All five methods produce a chair that a human would recognise, and every one of them looks markedly better than the 256-point input. Seat and back surfaces are re-filled and the leg structure is restored. On visual inspection alone one would conclude that recovery had succeeded.

What the classifier sees

Four of these five reconstructions classify worse than the 256-point cloud they were built from (−0.51 to −2.04 pp). Visual plausibility and PointNet++ discriminability are therefore not the same property — which is exactly why the quantitative protocol is necessary.

Sample: chair / chair_0890, test split. Interactive 3-D viewers for 10 shape-line pairs are available under [figures/modelnet40/pointcloud_examples_interactive_v2/dropdown_v2/](#)

SECTION

IV

Problems Encountered and Analysis

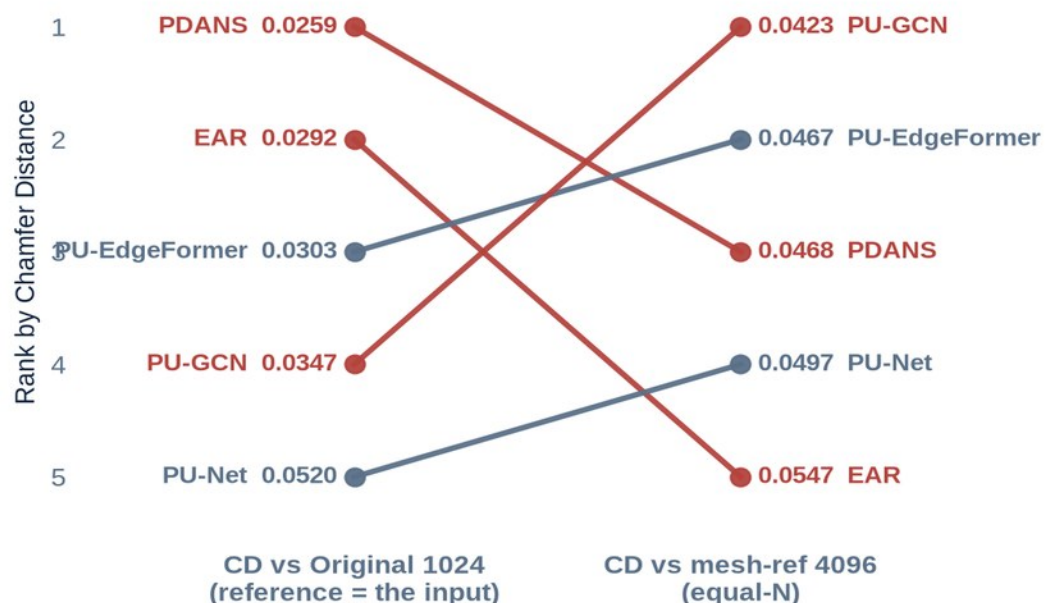
Defects found, how they were diagnosed, and what remains open

- A methodological defect that changed the conclusions: reference-frame dependence
- What the reference change moved, what it left bit-identical, and why that is the proof
- Protocol defects caught by audit: ratio, line definition, and silent resampling
- Metric reproducibility, dataset-build, evaluation-scope and documentation defects
- Cluster failures in generation and training, and the challenge raised on 2026-07-23

The Chamfer ranking reverses with the choice of reference set

Discovered on 2026-08-03 while auditing three geometry tables that all carried the name “final”.

Line A: the ranking is an artefact of the reference set



Reference set used	N	Verdict
Dense 10,000-pt mesh GT (table of 17 July)	12,311	<i>Superseded</i>
Original 1024 (unequal cardinality)	2,468	Biased — withdrawn
Mesh-ref 4096 equal-N (current protocol)	2,468	Adopted

TWO REFERENCES AGREE, ONE DISAGREES

The equal-N ranking reproduces the one obtained independently against a dense 10,000-point mesh ground truth — PU-GCN first, EAR last. Only the unequal-cardinality table contradicts both, and it is the one whose reference is also the methods' own input.

This defect became a result. PU-Net moves from 4th to last and EAR from last to 2nd purely as an artefact of the reference cardinality. The reversal is itself empirical evidence that unequal-cardinality CD is unsound, and it is reported in the thesis as a methodological contribution rather than quietly fixed.

What the reference change moved — and what it left untouched

Same shapes, same metric code, same 2,468-sample scope. Only the reference point set differs.

Line A method	CD vs Original 1024	CD vs mesh-ref 4096	CD rank	HD vs Original 1024	HD vs mesh-ref 4096	HD rank
PDANS	0.0259	0.0468	1 → 3	0.0838	0.1135	2 → 4
EAR	0.0292	0.0547	2 → 5	0.0989	0.1154	4 → 5
PU-EdgeFormer	0.0303	0.0467	3 → 2	0.0457	0.1109	1 → 2
PU-GCN	0.0347	0.0423	4 → 1	0.1019	0.0932	5 → 1
PU-Net	0.0520	0.0497	5 → 4	0.0881	0.1112	3 → 3

WHERE NOTHING MOVED

The Line B upsamplers were already being compared at 1024 against a 1024-point reference. Their CD values are unchanged by the new protocol — the two tables agree to the last representable digit: the largest relative difference is $2.2\text{e-}16$, and two of the five rows are bit-identical.

WHERE IT MOVED MOST

Every row whose cardinality was mismatched. Line A (4096 vs 1024) reorders completely; the Line B Downsampled ×4 branch (256 vs 1024) moves from CD 0.0464 to 0.1333 — a factor of 2.87 — once it is measured against a 256-point reference.

That contrast is the proof: the numbers moved exactly where the point counts disagreed, and nowhere else.

How the biased comparison came about

Not a coding error. A default that was reasonable when it was made and stopped being reasonable once Line A existed.

1

The dataset offered exactly one per-shape reference

The corpus was built as Original 1024 clouds. When geometry evaluation was first added, that file was the only ground truth available in per-shape correspondence, so it became the reference by default rather than by choice.

2

For Line A that reference is also the input

Line A upsamples Original 1024 to 4096. Measuring the output against Original 1024 therefore scores how little the method disturbed its own input — the inverse of what upsampling is for.

3

Line B was correct, and that masked it

Line B upsamplers emit 1024 points against a 1024-point reference that is genuinely not their input. Those rows were right all along. Half the table being sound made the whole table look self-consistent.

4

The wrong values still looked plausible

The bias does not produce absurd numbers. It produced well-behaved ones in the expected 0.02–0.05 band, with sensible standard deviations, so nothing in the output invited a second look.

5

A contradicting table existed but was never aligned

The dense 10,000-point mesh-GT table already ranked PU-GCN first, agreeing with what equal-N later found. It used a different sample scope (12,311 vs 2,468) and added P2F, so the disagreement sat unexamined between two files.

How it surfaced

On 2026-08-03 three geometry tables in reports/ all carried the name “final”. Auditing them side by side showed the Line A ranking reversing between two of them, which is what triggered the investigation.

GENERALISABLE LESSON

A reference set that is also the model input cannot measure that model. The check is cheap: feed the metric a degenerate output that does nothing. If it scores well, the metric is measuring the wrong quantity.

Protocol defects found and corrected before the final round

Each of these would have confounded the comparison. All were caught by audit rather than by a failing job.

Date	Defect	Why it would have confounded the result	Correction
2026-06-25	Main ratio was $R = \times 2$	The PU-family methods default to $\times 4$; a $\times 2$ main protocol would have compared every learned method outside its designed operating point.	Switched to $\times 4$. The completed $\times 2$ EAR run was reclassified as an ablation and kept, not deleted.
2026-06 (late)	Line B was 512 $\rightarrow$ 2048	Decimation and recovery used different factors, so “recovery” did not return to the original cardinality and could not be read against Original 1024.	Redefined to 256 $\rightarrow$ 1024, making a strict $\times 4$ decimation / $\times 4$ recovery closed loop.
2026-06-25	Step-6 baseline was resampled in the loader	The 512-point baseline was silently resampled to 1024 at load time, so it would have “improved” for reasons unrelated to any upsampler — the point-count effect wearing the upsampling effect’s clothes.	Demoted to a negative-control ablation. The canonical Line B baseline is native and never resampled.
2026-07-07	The dataloader could resample any branch	train_pointnet2.py defaults to --num-point 1024 and the loader resamples whenever the on-disk count differs, so a mis-launched branch would have been silently cropped or padded. The legacy train_pointnet2.sh wrapper still held old-protocol branches.	allow_resample: false in all 14 configs; every sbatch calls the .py directly and never the wrapper; a point-count audit verifies each branch.

Metric reproducibility and evaluation scope

PROBLEM 2 · OPEN, LOW SEVERITY

NUC is not exactly reproducible

The same Original 1024 point cloud yields $NUC = 1.080757 / 1.083553 / 1.061943$ in three separate runs, a spread of roughly 0.3 to 1.0 %. Cause: the 128 NUC query centres are drawn with a run-level RNG that is not bound to the per-sample seed, so a different subset of centres is selected each time.

Impact assessment: the between-method spread (PU-GCN 1.524 versus PU-Net 1.080, i.e. 0.44) exceeds this noise by roughly two orders of magnitude, so no reported conclusion changes. Fix: derive the NUC seed per sample, or average over repeated draws.

LOCATION

```
scripts/compute_geometry_equal_n.py:82-86
    rng = np.random.default_rng(seed)
    centers = points[rng.choice(n, size=k,
                               replace=False)]
:36 NUC_QUERY_CENTERS = 128
```

PROBLEM 3 · RESOLVED

Mismatched evaluation scopes

The earlier combined summary table placed geometric means computed over 12,311 shapes (train and test) in the same row as classification accuracies computed over the 2,468 test shapes only, and then discussed the two as though they described one population.

Resolution: the equal-N protocol restricts every geometric evaluation to the test split ($N = 2,468$), matching the classification scope exactly. All numbers in this deck use the unified scope.

WHY IT MATTERED

Any correlation drawn between a train-plus-test geometric mean and a test-only accuracy is not a correlation over a common set of shapes, and cannot support a causal reading.

General principle applied to both. Every defect is reported together with a quantified magnitude and an explicit statement of whether it changes any conclusion. Problem 2 does not — the noise is roughly 100 times smaller than the effect. Problem 3 did, and was corrected before any result in this deck was produced.

Dataset build, documentation drift, and repository hygiene

#	Problem	Evidence	Assessed impact	Status and action
4	Four mesh-reference shapes were reused rather than regenerated	Build audit: created = 12,307, reused = 4 (train/airplane/airplane_0005–0008). It cannot be confirmed that these four came from the current <code>stable_seed</code> formula.	None on any reported number: all four lie in the train split, while every geometric result is test-only. They do enter the mesh-ref training set.	Deferred deliberately — the two mesh-ref trainings were already reading the directory. Regenerate and retrain before submission.
5	README describes a protocol that its own tables contradict	README states “geometry reference: Original 1024 only, no mesh, no P2F”, while the table it links to reports <code>gt_point_count</code> = 10000 and <code>p2f_status</code> = exact.	No effect on results, but a live risk that a reader cites the wrong protocol.	Rewrite the protocol section to describe equal-N, and mark the two superseded geometry tables as superseded.
6	Large artefacts untracked and duplicated report copies	62 pending changes. <code>.gitignore</code> covers neither <code>*.tar.gz</code> (32 MB and 5 MB archives) nor <code>external/</code> nor the 8.4 MB per-sample CSV. <code>presentations/</code> duplicates files held in <code>reports/</code> .	No effect on results; risk that duplicate copies diverge and that large binaries are committed.	Extend <code>.gitignore</code> before the next commit; keep <code>reports/</code> as the single source and link to it from <code>presentations/</code> .

Upsampling-generation failures on the cluster

Producing 12,311 clouds per method per line, five methods, two lines. Every failure below was diagnosed in a written report before any re-run.

Failure	Symptom and root cause	Resolution	Outcome
PU-Net Line B smoke Job 1727073 2026-07-03	All 40 smoke samples failed with <code>FileNotFoundError</code> while <code>*saving*</code> — the per-class output directory was never created. Inference itself had run: the environment came up, TF custom ops recompiled (~15 min), 40 samples processed.	Added the missing <code>mkdir</code> to the output writer; the rerun passed 5/5 train and 5/5 test at the expected (1024, 3) shape with no NaN/Inf.	PASS
PU-Net Line B full Job 1729235 2026-07-05	11 of 16 array tasks hit the 24 h wall clock and one failed outright; each chunk had ~770 shapes to push through a TF1 inference path slower than budgeted. 9,150 of 12,311 outputs produced.	Chunked resume over the 3,161 missing shapes only, then a strict output audit.	12,311 / 12,311
PU-GCN Line B stall Job 1731794 2026-07-06	Tasks 0–12 completed in ~2.5 h on RTX 2080 Ti nodes. Tasks 13–15 sat <code>RUNNING</code> for 21 h 39 m on one RTX 3080 node with no <code>TIMEOUT</code> , <code>FAILED</code> or <code>OOM</code> recorded — a genuine hang inside the custom <code>knn_point_2</code> CUDA op. 10,010 / 12,311, 2,301 missing; <code>raw = strict</code> , so not a post-processing problem.	Cancelled for resume and relaunched with a resume manifest on a different node type.	Completed
PU-GCN resume Job 1732373 2026-07-06	All 8 array tasks failed within 23–51 s. GPUs were allocated correctly; this was a Python <code>SyntaxError</code> at import — <code>EXPECTED_INPUT_POINTS</code> was used in an <code>argparse</code> default (line 147) before its global declaration (line 160), so inference never started.	Removed the global statements, renamed the module constants, and passed the two values as explicit arguments into <code>process_row()</code> ; verified with <code>py_compile</code> .	Completed; 10,010 existing outputs untouched
PDANS environment	Jobs aborted immediately with <code>torch.cuda.is_available() == False</code> despite a GPU allocation — a mismatch between the loaded module set and <code>LD_LIBRARY_PATH</code> .	Aligned the module environment and library path inside the submission script.	Completed
TF1 custom CUDA ops (PU-Net, PU-GCN, PU-EdgeFormer)	The TF1 <code>tf_ops</code> kernels are highly sensitive to the compiler / CUDA toolkit / driver combination and had to be rebuilt after migrating from the lab machines to the HPC cluster. PU-GCN sat at status <code>pending_environment</code> until its TF 1.15 environment was validated.	Per-method build and runtime environments pinned; compatibility shim retained under <code>.cuda_runtime_compat/</code> .	Completed

Training-pipeline failures, and the audit that caught them

The smoke round exists precisely so that these surface on 10 short jobs rather than on 14 runs of 200 epochs.

SMOKE ROUND, 2026-07-07 — 8 OF 10 BRANCHES FAILED

Both baselines passed. Every one of the eight upsampled branches failed at dataloader initialisation with the identical error: `FileNotFoundError: Missing label map ... /metadata/class_to_idx.json`. The strict upsampling output directories had been written with `train/` and `test/` only, while `ModelNetNPYDataset` also requires a label map. No GPU work ever started, so nothing about the data itself was implicated.

Branch group	Jobs	Stage reached	Diagnosis	Fix
Line A baselines	1733475	completed	—	<i>none needed</i>
Line A upsampled ×4	1733476–79	dataloader init	<code>data_root</code> pointed at <code>strict_4N/</code> , which has no <code>metadata/</code>	Per-method <code>metadata/</code> with symlinked <code>class_to_idx.json</code> and <code>idx_to_class.json</code> — no <code>.npy</code> file touched
Line B baselines	1733159	completed	—	<i>none needed</i>
Line B upsampled ×4	1733160–63	dataloader init	same missing label map under <code>strict_N/</code>	Same symlink fix; manifests deliberately left absent so the loader falls back to a directory scan

Why it was worth fixing this way. Copying the point clouds into a loader-shaped directory would have duplicated 60 GB and created a second copy that could drift. Symlinking only the two label maps leaves exactly one copy of every `.npy` on disk, which the subsequent point-count audit then verifies branch by branch.

A challenge to the headline result, and how it was settled

Raised 2026-07-23. If it had been correct, the entire Line A conclusion would have been an artefact of the evaluation rather than a property of the data.

The question: “Were the Line A 4096-point branches evaluated with a PointNet++ that had been trained on 1024 points?” If so, the 0.32–1.41 pp deficits would measure a resolution mismatch between training and test, not the quality of the upsampled clouds — and the result would say nothing about upsampling at all.

#	Evidence checked	What was found
1	Branch configuration	Every Line A YAML sets num_point: 4096 and expected_point_count: 4096.
2	Job submission	Each full-training sbatch passes --num-point 4096 and no pretrained or resume checkpoint argument.
3	Training code path	train_pointnet2.py contains no torch.load, resume, pretrained or load_state_dict path; the model is built by get_model(...) from random initialisation.
4	Observed batch shape	Every train.log records a first batch of (24, 3, 4096).
5	Initial loss	First loss 3.68–3.75 against $\ln(40) = 3.689$ — consistent with a freshly initialised 40-way classifier, not a warm start.
6	Run independence	Separate jobs and output directories per method, each reaching epoch 200/200 with its own best_model.pth.
7	Run health	No NaN, OOM or CUDA traceback in any train log or SLURM stream.

Answer: no — every 4096 branch was trained from scratch at 4096. A clean re-run under a separate experiment name was judged unnecessary. The wording “PointNet++ trained only for 1024 points” is wrong and is avoided throughout this report.

Authenticity audit of the newest method before admitting it to the main tables

PU-EdgeFormer was integrated on 2026-07-16. Nine independent checks were run before any of its numbers were used.

Check	Question it answers	Line B	Line A	Result
Checkpoint restored	Were the published weights actually loaded, rather than a randomly initialised model?	YES	YES	PASS
Model code on the call path	Was the PU-EdgeFormer graph executed, verified from the SLURM logs and the call path?	YES	YES	PASS
Not a repeat ×4	Are the outputs genuinely new points, or each input point duplicated four times?	YES	YES	PASS
Not a copy of the input	Exact and rounded match ratios measured against the input cloud.	YES	YES	PASS
Not PU-GCN output	Are these files distinguishable from the PU-GCN outputs in the neighbouring directory?	YES	YES	PASS
No smoke-test leakage	Did any file from the earlier small-scale smoke run survive into the full output set?	YES	YES	PASS
Outputs fresh	Do the file timestamps correspond to the full production run?	YES	YES	PASS

Quantitative outcome. Across 200 sampled shapes per line: exact-match ratio 0.000000, duplicate ratio 0.000000, unique output count exactly 1024 in Line B and 4096 in Line A, and zero exact repeat or tile hits. The fail conditions — match ratio above 0.95, duplicate ratio above 0.60, unique count equal to the input count — triggered on 0 samples. Final verdict: TRUSTED, cleared for geometry metrics and for PointNet++ training.

SECTION



Status, Limitations and Outlook

What is settled, what is provisional, and what comes next

- How the protocol evolved, and which earlier configurations became ablations
- Limitations that must be stated in the thesis
- Conclusions and the remaining work before submission

Protocol evolution and the corrections behind it

Date	Change	Reason	Current status
2026-06-15	modelnet40_experiments structure and method_registry established	Asset indexing across methods and lines	Retained
2026-06-20	Main pipeline modelnet40_pointnet2_upsampling created	Consolidation onto a single project root	Current root
2026-06-25	Primary upsampling ratio changed from $\times 2$ to $\times 4$	Aligns with the default ratio of the PU-family methods and with the aims of the study	In force; $\times 2$ kept as ablation
2026-06 (late)	Line B redefined from downsampled50 (about 512 pts) to 256 $\rightarrow$ 1024	Creates a strict $\times 4$ decimation and $\times 4$ recovery closed loop	Canonical
2026-07-16	PU-EdgeFormer added as the fifth method	Covers the transformer-hybrid line of work; admitted only after the nine-check audit	In main tables
2026-08-03	Equal-N geometry protocol introduced; mesh references built at 256 and 4096	Response to the reference-frame defect (Problem 1)	Primary geometry protocol
2026-08-04	Mesh-ref classifiers trained; dense-mesh and P2F narrative removed from the presentation	Answers RQ4 and eliminates a superseded evaluation story	Current reporting basis

Demoted to ablations, not deleted: the $\times 2$ EAR Line B (512 $\rightarrow$ 1024), the naive resample baseline from Step 6, the downsampled50 512-point protocol, unequal-cardinality CD/HD rankings, and the dense-mesh exact-P2F narrative. Each is retained with its data so that the thesis appendix can show what was tried and why it was rejected.

What this study does not establish

Single downstream task and architecture

All conclusions concern PointNet++ SSG on ModelNet40 shape classification. Whether the same decoupling appears for DGCNN, Point Transformer, segmentation or detection is untested. The KITTI / CenterPoint detection line exists in the project but is not part of this analysis.

Upsamplers are used as released

Every learned method runs from its published checkpoint, trained on PU1K or PU-GAN. None was fine-tuned on ModelNet40, so part of every deficit may reflect a domain gap between the upsampler's training data and CAD shapes rather than a limitation of the method itself.

No validation split, by benchmark convention

Best Overall Accuracy is the maximum test accuracy over 200 epochs. This is the standard ModelNet40 protocol and it is applied identically to all 14 runs, but it is optimistic in absolute terms and should not be quoted as a clean held-out estimate.

Single seed per configuration

Each branch was trained once, with seed 42. Differences of a few tenths of a percentage point — notably the +0.42 pp PU-Net gain and the +0.05 pp mesh-ref gain — lie in a range where run-to-run variance cannot be excluded without repeated seeds.

One ratio, one dataset

Only $\times 4$ on ModelNet40 is examined. Behaviour at $\times 2$ or $\times 16$, on scanned rather than CAD data, or under input noise, lies outside the scope of the present claims.

What can now be written down

- 1 Point cloud upsampling does not generally improve PointNet++ classification accuracy on ModelNet40.
- 2 Under densification (1024 $\rightarrow$ 4096) all five methods fall below the native Original 1024 baseline of 91.95 %.
- 3 Under recovery (256 $\rightarrow$ 1024) only PU-Net exceeds the sparse baseline, by +0.42 pp, and stays 0.68 pp below Original.
- 4 Geometric metrics and task utility are misaligned: PU-GCN attains the best Chamfer Distance in both lines yet is outperformed on accuracy by PU-Net.
- 5 A mesh-sampled 4096-point control beats Original 1024 by only +0.05 pp, so point count alone explains neither the gains nor the losses.
- 6 Chamfer rankings depend on the cardinality of the reference set; equal-cardinality evaluation is therefore a prerequisite for any comparison.
- 7 Consequently, upsampling methods should report geometric quality and downstream utility separately, and should state whether they densify or recover.

Next steps: regenerate the four reused mesh-reference shapes and retrain the mesh-ref classifiers · bind the NUC seed per sample · reconcile the README and mark superseded tables · decide on repeated seeds for the headline Δ values

Material index — results, geometry and mechanism

158 reports exist under reports/; these are the ones this deck draws on directly. All paths are relative to the project root.

Purpose	Path
Classification results, all 14 runs	pointnet2_results/x4_two_line_final/**/metrics.json
Two-line classification summary	reports/modelnet40_pointnet2_final_two_line_classification_summary_with_pu_edgeformer.{md,csv}
Equal-N geometry (summary, per line, per sample, audit)	reports/modelnet40_geometry_equal_n_{summary,lineA,lineB,per_sample}.csv · _audit.md
Equal-N protocol definition	reports/modelnet40_geometry_equal_n_protocol.md
Cardinality-bias probe (fwd/bwd split, repeat ×4 control)	reports/modelnet40_geometry_cardinality_bias_probe.{md,json}
SA1 reception probe (what the layer receives)	reports/modelnet40_sa1_reception_probe.{md,json}
Superseded geometry table (unequal cardinality)	reports/modelnet40_geometry_vs_original_summary.json
Mesh-ref classification control	reports/modelnet40_pointnet2_mesh_ref_baseline_results.{md,csv} · _status.md
Mesh-ref build audits	reports/modelnet40_mesh_ref_{256,4096}_build_audit.{md,csv}
Project figures used in this deck	figures/modelnet40/original_reference_revised/ · .../pointclouds/
Interactive 3-D viewers, 10 shape-line pairs	figures/modelnet40/pointcloud_examples_interactive_v2/dropdown_v2/
Full experimental dossier	presentations/ModelNet40_Thesis_Complete_Dossier.{md,pdf}

Material index — problem diagnoses, audits and scripts

One written diagnosis per problem, produced before the corresponding re-run was launched.

Problem / purpose	Path
P1 Reference-frame defect	reports/modelnet40_geometry_equal_n_audit.md · _protocol.md
P2–3 NUC reproducibility, evaluation scope	reports/modelnet40_geometry_equal_n_meta.json · scripts/compute_geometry_equal_n.py
P4 Mesh-ref build reuse	reports/modelnet40_mesh_ref_{256,4096}_build_audit.{md,csv}
P5–6 Documentation drift, repository hygiene	README.md · .gitignore
P7 Protocol corrections (ratio, line B, resampling)	reports/modelnet40_protocol_x4_correction_note.md · step8_protocol_audit_*.md
P7 Dataloader point-count audit	reports/modelnet40_pointnet2_dataloader_pointcount_audit.md
P8 Generation failures (PU-Net, PU-GCN, PDANS)	reports/modelnet40_lineB_punet_*_diagnosis.md · modelnet40_pugcn_lineB_stall_diagnosis.md
P8 PU-GCN resume SyntaxError	reports/modelnet40_lineB_pugcn_resume_1732373_failure_diagnosis.md · _script_fix_smoke.md
P9 Training smoke failures and metadata fix	reports/modelnet40_pointnet2_line{A,B}_smoke_failure_diagnosis.md · _metadata_fix_audit.md
P10 Answer to the 2026-07-23 challenge	reports/modelnet40_pointnet2_lineA_4096_retraining_audit_for_teacher.md
PU-EdgeFormer authenticity (9 checks)	reports/pu_edgeformer_modelnet40_authenticity_verification_20260716.md
Geometry and mechanism scripts	scripts/{compute_geometry_equal_n,compute_cardinality_bias_probe,compute_sa1_reception_probe}.py
Dataset and training scripts	scripts/{build_mesh_reference_points,train_pointnet2}.py